CAPSTONE / END-TO-END LLM APPLICATION

GearMate

An evidence-backed assistant for student equipment lending

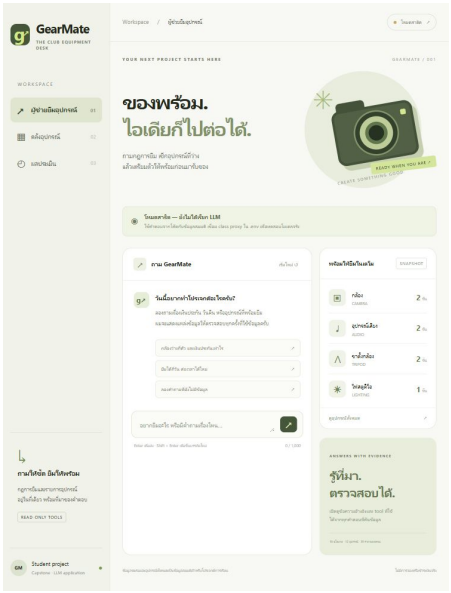
Author: Chayapon8251 | Repository: github.com/Chayapon8251/llm_lastwork

STATUS	IMPLEMENTATION AND OFFLINE EVALUATION COMPLETE
Live class proxy	Not yet verified: credentials unavailable at this report build
Data	Synthetic club policies and inventory; no real bookings
Purpose	A working, inspectable project foundation, not a final submission claim

1 / Problem and intended users

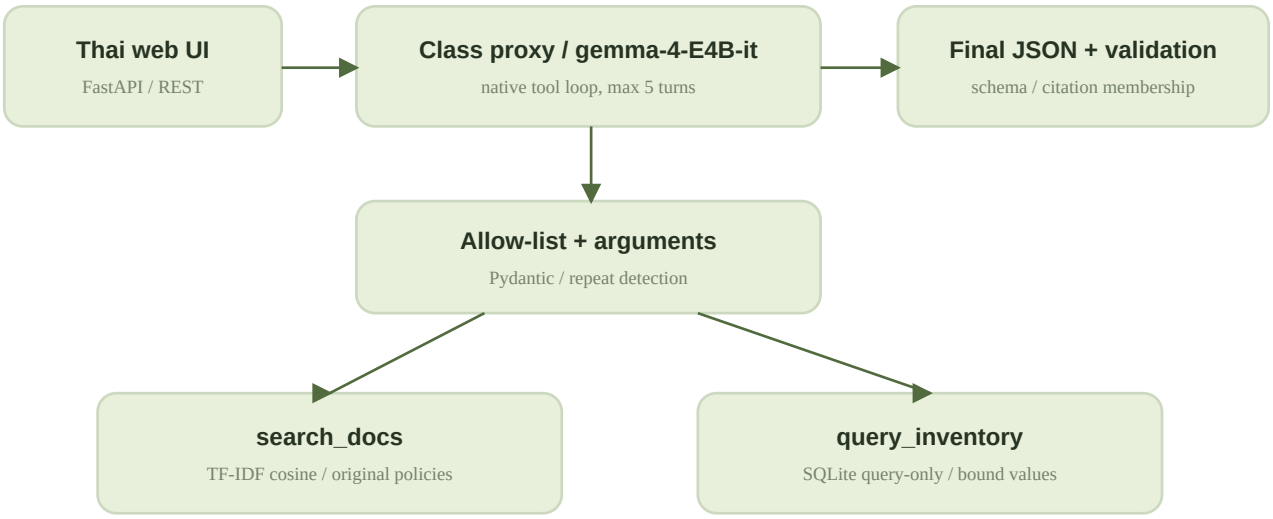
Students preparing a club production need both policy information and available equipment. A question such as "How many cameras are available, and what deposit is required?" spans prose and structured records. A language model should not invent a stock count or a borrowing rule.

GearMate combines a Thai web interface, document retrieval and a read-only SQLite tool. A live agent chooses which tool to call, sees its results, and returns a validated answer with source IDs. A separately labelled deterministic demo keeps the interface inspectable without credentials.



Interface screenshot from offline-demo mode. The banner explicitly states that no LLM was called.

Two data sources. One bounded loop.



Tool results return to the model with tool_call_id

Live execution sequence

1. FastAPI validates the user input and bounded history.
2. OpenAI() calls the class proxy with model gemma-4-E4B-it and tool schemas.
3. The model requests a tool; code validates its name and arguments.
4. Tool output returns as role: tool, matched by tool_call_id; the model continues.
5. A normal response with no tool call ends the loop. A separate request renders JSON, which Pydantic and citation-membership checks validate.

Course family	Concrete implementation
Structured output	Validated answer / source_ids / supported JSON; one repair attempt
Evaluation (required)	30 labelled retrieval questions; four reproducible comparisons
Agent loop (agentic)	Native function calling; no-tool stop, max turns, call budget
Advanced SQL (agentic)	Typed filters compile to bound SELECT on a query-only database
Served interface	FastAPI endpoints, Thai chat, stock view, metrics view

Retrieval is lexical: character-trigram TF-IDF vectors, not neural semantic embeddings. The SQL tool uses fixed templates rather than unconstrained text-to-SQL. These are deliberate scope choices. No sandbox code execution, web browsing or MCP server is claimed.

A small corpus with explicit provenance.

The repository commits 18 topic-sized policy records in Thai with English translations, and 12 synthetic equipment records. Topics include eligibility, standard duration, deposits, late fees, reservations, training, damage and privacy. Each policy has a stable ID; source links identify the data file and record. This is an original hypothetical domain, not the course reference corpus.

Chunking and indexing

One policy record is one chunk, preserving conditions such as the 24-hour extension deadline and staff approval requirement. TF-IDF weights normalized character trigrams and compares vectors by cosine. BM25 uses English tokens and Thai bigrams. Both indexes are built in memory from the committed corpus; there is no external embedding service or persistent vector database.

Candidate systems

System	What changes
Vector top-1	Naive named baseline: one cosine-ranked chunk
Vector top-3	Budget control: same retriever, three context chunks
BM25 top-3	Keyword ranking, same output budget
Hybrid top-3	RRF(60) over top ten from each ranking, then three results

Threshold and safety boundaries

All candidates use a cosine floor of 0.04. This is a development heuristic, not a calibrated confidence score. A low threshold can admit off-topic text, especially with Thai character overlap. A fresh negative benchmark is needed before relying on automatic abstention. The final model must judge whether evidence answers the question.

Inventory records are loaded into an in-memory SQLite database and the connection is switched to query_only. The only tool accepts category, item code and status. Field names are fixed by code and values use bound parameters. No endpoint accepts arbitrary SQL or writes a reservation.

Measure the change, including the trade-off.

The curated test set contains 30 questions: 18 require one source and 12 require two. It includes Thai questions, exact item codes and English multi-source queries. Six development questions and six unsupported/manual scenarios are separate. Labels were authored with AI assistance alongside the corpus and are not independently human-validated. This creates lexical-overlap and selection bias.

Metrics

Macro recall@k = relevant IDs in the returned top-k / relevant IDs. Precision@k divides by k. MRR is the mean reciprocal rank of the first relevant hit in the returned list, or zero if absent. Complete coverage is the fraction of questions retrieving every required source. Each result uses its actual cutoff.

System	k	Recall	Precision	MRR	Complete
vector top1	1	0.8000	1.0000	1.0000	0.6000
vector top3	3	0.9833	0.4556	1.0000	0.9667
bm25 top3	3	0.9667	0.4444	0.9833	0.9333
hybrid top3	3	0.9833	0.4556	1.0000	0.9667

What the numbers support

Increasing the output from one to three vector results raises recall from 0.8000 to 0.9833 (+18.33 percentage points) and complete coverage from 0.6000 to 0.9667. Precision falls from 1.0000 to 0.4556 because more context includes irrelevant chunks. The gain is partly an output-budget change, not proof that a new retrieval algorithm is better.

At the same k=3, hybrid matches the vector results rather than improving them. The shipped default is therefore vector top-3. Hybrid remains an evaluated alternative. MRR=1.0 means the first relevant hit was first on this small set; it does not mean every needed source was retrieved.

Reproduce: python -m scripts.evaluate. results/retrieval.json records rankings, subgroup metrics, failures and newline-normalized SHA-256 hashes. No model calls contribute to this table. These are retrieval metrics, not answer correctness, faithfulness or real-user success rates.

A high average still misses important context.

Failure A: multi-source retrieval

Test t27 asks: "camera deposit and first use training requirements". Gold sources are camera_deposit and training. Vector top-3 instead returns training, lighting_deposit and audio_deposit. Recall for this question is 0.5. MRR remains 1.0 because training is first. Similar vocabulary across deposit policies distracts the lexical retriever.

Failure B: an unseen short question

The post-implementation question "How many days can I borrow?" in Thai retrieves extension, membership and weekend_return, missing the standard-duration policy. The more explicit Thai query "standard borrowing duration" succeeds. This is an observed failure, not a removed test. Model-driven query refinement may help, but that has not yet been demonstrated against the real class proxy.

Failure C: unavailable knowledge or action

Drone inventory, current weather and next month's university holidays are not represented. Booking is a write action and is intentionally unavailable. The damage policy has no flat compensation amount. A correct agent must acknowledge missing data instead of fabricating a fee, date or completed transaction.

What was actually verified

Check	Evidence / scope
Automated tests	39 passing tests: validators, query-only DB, injection values, metrics, REST and agent protocol
Protocol tests	Stubbed SDK messages verify tool_call_id, parallel calls, repeat blocking, caps and JSON repair; not an LLM benchmark
Browser checks	Chat, SQL+policy example, citation expansion, trace, evaluation view and mobile stock filter
Real proxy / faithfulness	Not yet run; requires class credentials and manual answer review

Citation membership guarantees only that an ID was observed in a tool result. It does not prove that every sentence follows from that source. Prompt labels discourage injection but the stronger boundary is architectural: no arbitrary execution or write tool exists.

Ready to inspect. Not yet ready to submit.

Remaining completion work

1. Configure OPENAI_API_KEY and OPENAI_BASE_URL from the instructor; never commit them.
2. Run python -m scripts.smoke_live using gemma-4-E4B-it. Inspect success and failure answers, tool selection, citations and latency.
3. Have the student verify the corpus and labels, then collect a fresh test set from an independent person.
4. Add a manually reviewed answer-quality evaluation. Separate unsupported claims, wrong answers and reasonable abstentions.
5. Update this report with actual live results and record a narrated 3-6 minute demo showing one success and one honest failure.

Boundaries of the current evidence

The corpus is synthetic and small. Test questions were created with the same source material and the benchmark has now been inspected, so it should be considered a development benchmark going forward. The lexical representation is not a neural encoder and cannot reliably match arbitrary paraphrases. The default threshold has not been calibrated on a large negative set. Inventory is a fixed snapshot with no real accounts or operational integration.

What I would improve next

Collect independent labels before adding complexity. Then compare multilingual neural embeddings, query construction and a cross-encoder at the same context budget. Keep only changes that improve the relevant metric enough to justify latency and cost. Add authentication, operational logs with privacy controls and real inventory updates only if the prototype becomes a deployed service.

Submission map

Deliverable	Current artifact / remaining gap
Q1 repository	github.com/Chayapon8251/llm_lastwork; run instructions, original data, metrics and tests
Q2 report	This six-page progress report; regenerate after real-proxy verification
Q3 demo URL	Narration outline supplied in docs/demo-script-th.md; recording and upload remain

References and authorship

Course Lectures 8-14 supplied the concepts of retrieval metrics, RRF, tool round-trips, bounded loops and SQL guardrails. Native calling follows the official OpenAI guide: https://developers.openai.com/api/docs/guides/function-calling. Code and synthetic corpus were created for this project with AI assistance. The student must inspect, adapt and defend the work; this report does not claim independent human annotation or real LLM performance.